



UNIVERSITÀ DEGLI STUDI DI TRIESTE

Progetto di Basi di Dati
Gestione Ferroviaria R&F

IN0500767

Pellizzaro Federico

 GITHUB REPOSITORY

 WEBSITE

Indice

I Introduzione

- i Requisiti per la Base di Dati
- ii Azioni richieste dal Committente

II Progettazione

- i Diagramma Entity – Relationship
- ii Dizionario dei Dati
- iii Vincoli non esprimibili
- iv Tabella dei volumi
- v Analisi delle Ridondanze
- vi Eliminazione Generalizzazioni
- vii Partizionamenti/Accoppiamenti
- viii Scelta Identificatori
- ix Diagramma Entity – Relationship ristrutturato
- x Passaggio al Modello Relazionale
- xi Schema Logico
- xii Normalizzazione

III Realizzazione Base di Dati e Operazioni

- i Creazione DataBase
- ii Operazioni SQL

I Introduzione

La Società Ferroviaria R&F (*Regionali & Freccie*) deve gestire i collegamenti ferroviari fra le città del Nord Italia. L'intera gestione di questo progetto non prevede solo un'implementazione della gestione dei treni ma prevede anche la gestione dei biglietti che i futuri clienti compreranno.

Requisiti per la Base di Dati

Per quanto riguarda i requisiti richiesti dalla società, un *treno* è composto normalmente da 2 *locomotive*, e da *vagoni passeggeri* aggiuntivi (6 vagoni di norma). Una locomotiva è caratterizzata da un gruppo invece i vagoni passeggeri sono caratterizzati dal numero di posti disponibili che sono all'interno del vagone. Ogni vagone però, che sia locomotiva o vagone passeggero è caratterizzato da un peso (misurato in tonnellate) e dalla posizione nel treno. Ogni treno che sia Regionale o Freccia, percorrerà durante il giorno sempre la stessa *tratta* la quale collega 2 *stazioni* del Nord Italia, per ogni tratta si vuole tenere traccia della lunghezza della tratta, invece per quanto riguarda le stazioni basta il nome della stazione. Un *cliente* è caratterizzato da un nome, cognome, ed email (facoltativa siccome non tutti comprano i biglietti online). Un cliente può comprare i *biglietti* del treno i quali devono avere un costo associato. Per ogni acquisto di un biglietto deve essere segnata la data di acquisto di tale biglietto e la data dell'effettivo viaggio. Per ogni biglietto, sono associate le informazioni relative alle *corse* che il cliente deve prendere, in merito alle corse, sono previste solo corse in giornata. Siccome i treni effettuano anche fermate intermedie durante il tragitto, per corsa si intende una parte della tratta percorsa dal treno oppure l'intera tratta percorsa dal treno, ed è composta dal treno che bisogna prendere, dall'orario di partenza dalla stazione di inizio corsa, e dall'orario di arrivo alla stazione di scalo per prendere un'altra corsa oppure alla stazione di fine. Come esempio, considerando la tratta “*Venezia S. Lucia – Trieste Centrale*” una corsa può essere il tragitto tra “*Trieste Airport*” e “*Monfalcone*”. La società è interessata a rilevare solo l'ultimo acquisto fatto da un cliente, per ogni tipo di biglietto acquistato dal suddetto cliente, in modo tale da non sovraccaricare i database. Nel caso in cui un cliente in un dato periodo compri 2 biglietti uguali, sarà tenuta traccia nel database principale solo dell'ultimo acquisto fatto. Invece il precedente acquisto sarà salvato in un altro database di “storage” non gestito in questo progetto.

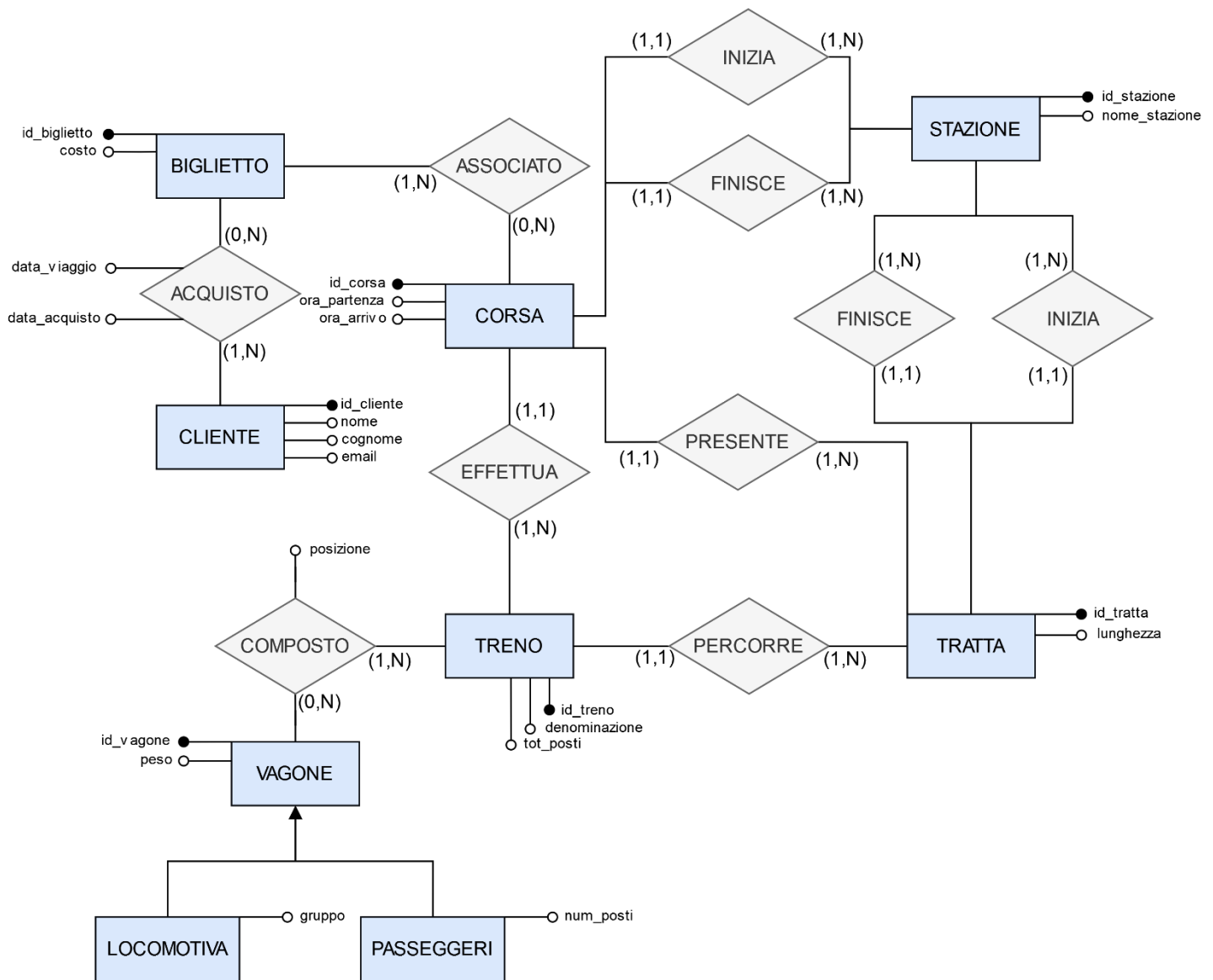
Osservazioni: è importante specificare che un biglietto non è strettamente vincolato ad un cliente, tanti clienti possono comprare uno stesso biglietto, come anche nessun cliente può comprare un determinato biglietto. I biglietti esistono a prescindere che qualcuno li compri oppure no.

Azioni richieste dal committente

1. Dato l'inserimento di un mese, si vuole avere il numero totale di biglietti venduti mensilmente (dell'anno corrente) raggruppati per giorni (interattiva/1 volta al mese).
2. Avere una lista delle stazioni più trafficate nell'arco dell'anno (batch/1 volta per anno).
3. Dato un treno, si vuole avere una tabella con tutte le fermate e il rispettivo orario della fermata che il treno effettuerà (interattiva/100 volte al giorno).
4. Per l'acquisto di un biglietto che coinvolga solo una corsa, premesso l'inserimento della "stazione di destinazione e di partenza", si vuole avere un elenco di tutti i treni che partiranno dalla stazione di partenza inserita fino alla stazione di destinazione inserita (interattiva/1000 volte al giorno).
5. Si vuole avere una tabella con tutti i treni e per ogni treno il numero totale di posti disponibili nel treno (batch/10 volte al mese).
6. Si vuole avere un riepilogo dettagliato del biglietto comprato, con nome e cognome dell'utente, costo del biglietto, data viaggio, e informazioni relative alle corse che il cliente deve prendere (interattiva/10 volte al giorno).

II Progettazione

Diagramma Entity – Relationship



Osservazione : con i pallini neri sono stati denotati gli identificatori.

Dizionario dei Dati

Dizionario delle *Entità*

Entità	Descrizione	Attributi	Identificatore
Passeggeri	Vagone adibito per i passeggeri del treno	id_vagone peso num_posti	id_vagone

Locomotiva	Vagone che fa muovere il treno	id_vagone peso gruppo	id_vagone
Treno	Veicolo di trasporto composto da vagoni	id_treno denominazione tot_posti	id_treno
Tratta	Inizio e fine del collegamento percorso dal treno	id_tratta lunghezza	id_tratta
Stazione	Punto di partenza o fine o di scalo di un treno	id_stazione nome_stazione	id_stazione
Corsa	Tutto o parte della tratta percorso da un treno	id_corsa ora_partenza ora_arrivo	id_corsa
Biglietto	Documento al quale è associato un costo e le relative corse riferite al biglietto	id_biglietto costo	id_biglietto
Cliente	Persona fisica che compra il biglietto/i	id_cliente nome cognome email	id_cliente

Dizionario delle *Relazioni*

Relazione	Descrizione	Componenti	Attributi
Composto	Associazioni dei vagoni ai rispettivi treni	Vagone Treno	posizione
Percorre	Ogni treno percorre una tratta	Treno Tratta	
Inizia	Dove la tratta/corsa iniziano	Tratta Corsa Stazione	
Finisce	Dove la tratta/corsa finisce	Tratta Corsa Stazione	

Presente	In una tratta sono presenti diverse corse	Corsa Tratta	
Effettua	Un treno che percorre una tratta può effettuare varie corse	Corsa Treno	
Associato	Ad un biglietto può essere associata solo una o più corse	Corsa Biglietto	
Acquisto	Un biglietto è acquistato da un cliente	Biglietto Cliente	data_acquisto data_viaggio

Vincoli non esprimibili

Dal testo sorgono i seguenti vincoli non esprimibili :

- Durante l'inserimento di una nuova corsa non si può immettere un orario di arrivo inferiore all'orario di partenza oppure un orario di arrivo dopo mezzanotte.

Tabella dei volumi

Concetto	Tipo	Volume
Vagone Passeggeri	E	1.200 (200*6)
Vagone Locomotiva	E	400 (200*2)
Treno	E	200
Tratta	E	100
Stazione	E	1.000
Corsa	E	8.000
Biglietto	E	5.000
Cliente	E	20.000.000
Composto	R	1.600 (1.200+400)
Percorre	R	200
Inizia Tratta	R	100
Finisce Tratta	R	100
Inizia Corsa	R	8.000
Finisce Corsa	R	8.000
Presente	R	8.000

Effettua	R	8.000
Associato	R	8.000
Acquisto	R	25.000.000

Analisi delle Ridondanze

Per quanto riguarda le ridondanze, si può notare che nell'entità *treno* vi è l'attributo "tot_posti", il quale però può essere facilmente calcolato siccome per ogni *vagone passeggeri* è specificato il numero di posti nel vagone. Quindi l'attributo "tot_posti" può essere eliminato. In tal modo non avrò il problema nella *3NF* per quanto riguarda i campi calcolati. Inoltre si può notare dal *Diagramma Entity – Relationship* che vi è un ciclo che coinvolge le entità *corsa*, *treno*, *tratta*. Si può sapere a quale tratta è riferita una corsa tramite il treno che percorre la suddetta tratta e quindi si può eliminare la relazione "presente" che collega *corsa* a *tratta*.

Eliminazione Generalizzazioni

Le generalizzazioni presenti riguardano il tipo di vagone del treno. Siccome una generalizzazione non sarebbe esprimibile in uno *Schema Logico* si è pensato di mettere un attributo "tipo" all'entità *vagone*, che identifica il modello di vagone. Il tipo può essere una parola composta da numeri (*vagone passeggeri*) oppure lettere (*vagone locomotiva*). Gli attributi di *passeggeri* e *locomotiva* saranno integrati tra gli attributi di *vagone*. Questa modifica genera automaticamente un nuovo vincolo non esprimibile :

- Per i vagoni *passeggeri* l'attributo gruppo sarà NULL, invece per i vagoni *locomotiva* l'attributo num_posi sarà NULL.

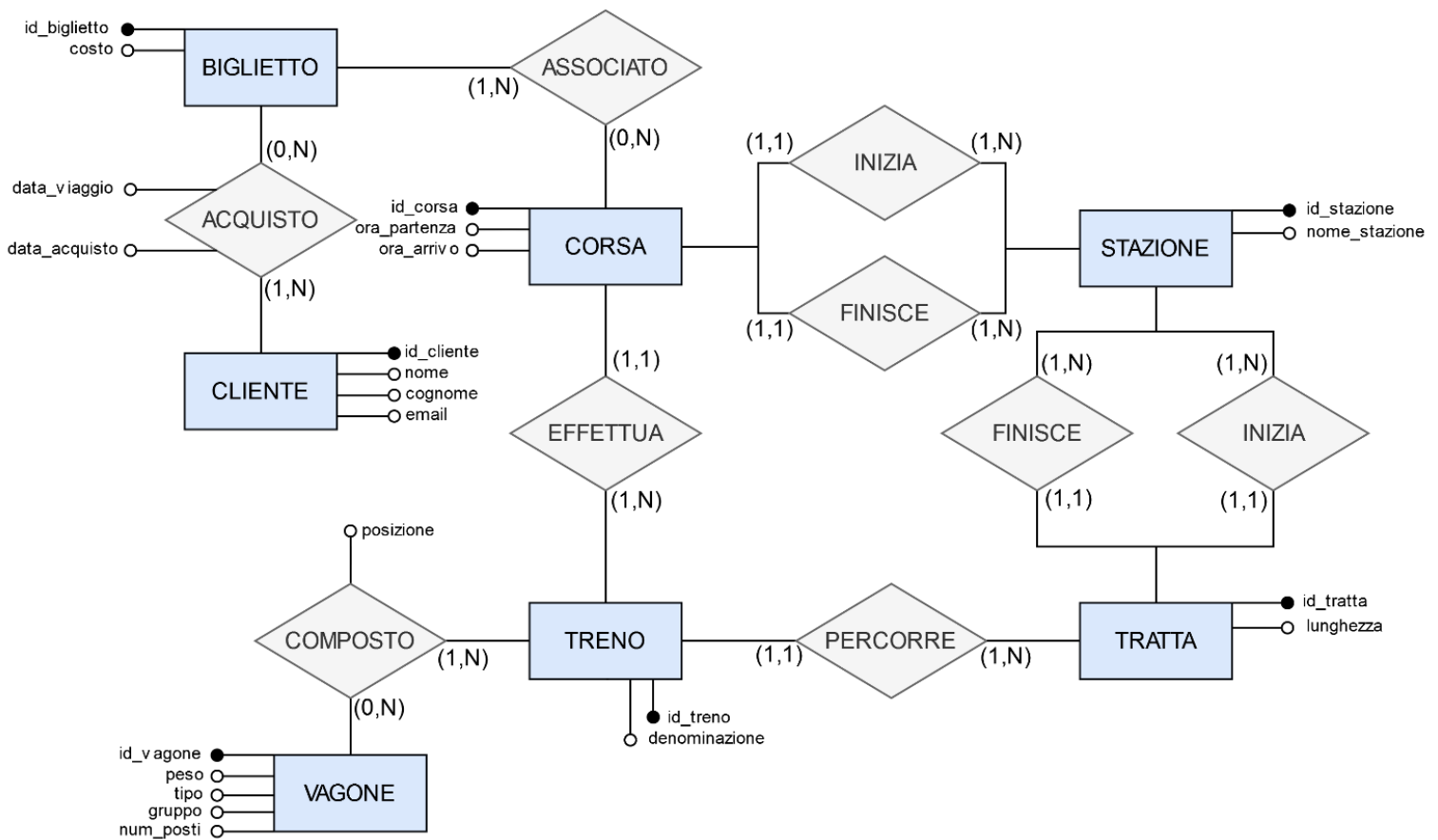
Partizionamenti/Accorpamenti

Non vi è la necessità di fare partizionamenti o accorpamenti tra le entità.

Scelta Identificatori

Per ogni entità presente è stato scelto di utilizzare un *id "nome_entità"* come identificatore. Questo *id* è un codice di 5 caratteri che possono variare tra cifre e lettere.

Diagramma Entity – Relationship ristrutturato



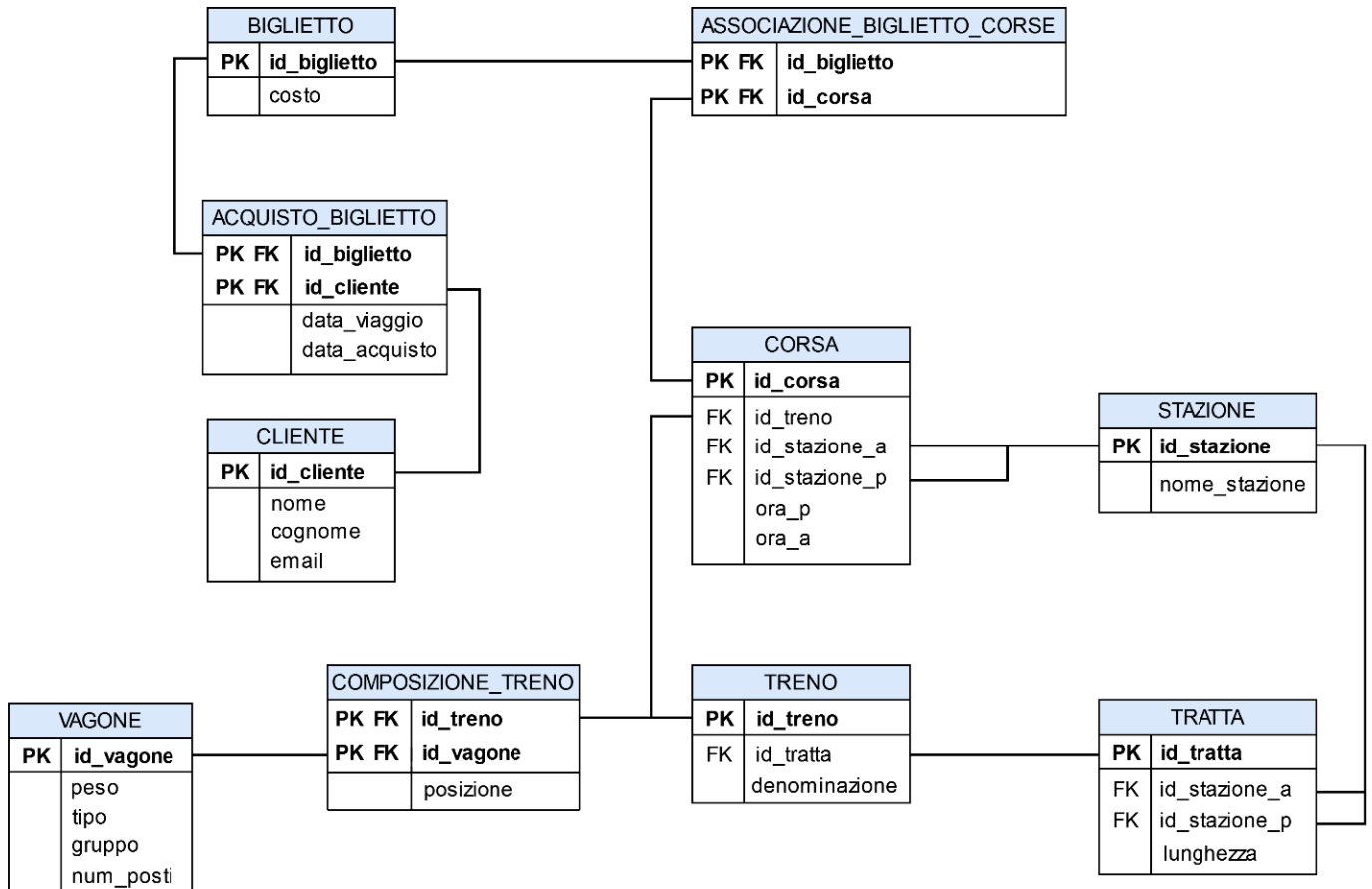
Passaggio al Modello Relazionale

Si passa così allo *Schema Logico* che sarà identificato dalle seguenti tabelle :

- **Stazione** (id_stazione , nome_stazione)
- **Tratta** (id_tratta, id_stazione_p , id_stazione_a, lunghezza)
- **Treno** (id_treno, id_tratta, denominazione)
- **Composizione_Treno** (id_treno, id_vagone, posizione)
- **Vagone** (id_vagone, peso, tipo, gruppo, num_posti)
- **Corsa** (id_corso, ora_p, ora_a, id_treno, id_stazione_p, id_stazione_a)
- **Associazione_Biglietto_Corse** (id_biglietto, id_corso)
- **Biglietto** (id_biglietto, costo)

- **Cliente** (id_cliente, nome, cognome, email)
- **Acquisto_Biglietto** (id_biglietto, id_cliente, data_acquisto, data_viaggio)

Schema Logico



Normalizzazione

1NF

La 1NF è rispettata in quanto non ci sono colonne che contengono valori multipli. Ogni colonna è atomica.

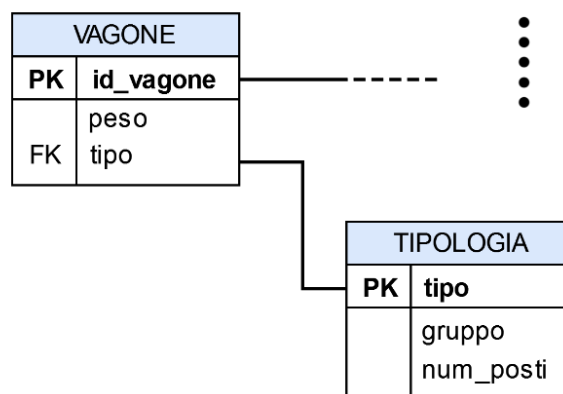
2NF

Anche la 2NF è rispettata perché è 1NF e inoltre ciascuna colonna di ogni tabella dipende esclusivamente dalla PK della relativa tabella. Cioè ogni colonna di ogni tabella dipende in senso stretto dalla PK della relativa entità di cui fa parte.

3NF

La 3NF invece non è rispettata in quanto esistono attributi di entità che non dipendono esclusivamente dalla PK dell'entità di cui fanno parte :

- *Vagone* : per questa entità l'attributo gruppo e num_posti dipendono dall'attributo tipo in quanto come detto in precedenza se tipo è una sequenza composta da numeri gruppo = NULL invece se è composta da lettere num_posti = NULL. Come possibile soluzione allora si può spezzare l'entità *vagone*, creando una nuova entità *tipologia* nella quale l'attributo tipo sarà la PK invece gruppo e num_posti saranno gli attributi della entità. Nell'entità *vagone* rimarrà solo l'attributo tipo :



Nel *Modello Relazionale* sarà fatta la seguente modifica :

- **Vagone** (id_vagone, peso, tipo)
- **Tipologia** (tipo, gruppo, num_posti)

Si potrebbe pensare che anche l'attributo email dell'entità *cliente* dipenda non solo esclusivamente dalla PK ma anche da nome e cognome, ma in questo caso non è così siccome un'email può non essere dipendente dal nome e cognome della persona che la utilizza.

III Realizzazione Base di Dati e Operazioni

Creazione DataBase

```
CREATE DATABASE gestioneFerrovie;
```

```
USE gestioneFerrovie;
```

```
CREATE TABLE Stazione (  
    id_stazione VARCHAR(5) PRIMARY KEY NOT NULL ,  
    nome_stazione VARCHAR(30) NOT NULL  
);
```

```
CREATE TABLE Tratta (  
    id_tratta VARCHAR(5) PRIMARY KEY NOT NULL ,  
    id_stazione_1 VARCHAR(5) NOT NULL ,  
    id_stazione_2 VARCHAR(5) NOT NULL ,  
    lunghezza INT(11) NOT NULL ,  
    FOREIGN KEY (id_stazione_1) REFERENCES Stazione(id_stazione),  
    FOREIGN KEY (id_stazione_2) REFERENCES Stazione(id_stazione)  
);
```

```
CREATE TABLE Treno (  
    id_treno VARCHAR(5) PRIMARY KEY NOT NULL ,  
    id_tratta VARCHAR(5) NOT NULL ,  
    denominazione VARCHAR(30) NOT NULL ,  
    FOREIGN KEY (id_tratta) REFERENCES Tratta(id_tratta)  
);
```

```
CREATE TABLE Tipologia (  
    tipo VARCHAR(5) PRIMARY KEY NOT NULL ,  
    gruppo VARCHAR(5),  
    num_posti INT(2)  
);
```

```
CREATE TABLE Vagone (  
    id_vagone VARCHAR(5) PRIMARY KEY NOT NULL ,
```

```
    peso INT(11) NOT NULL ,
    tipo VARCHAR(5) NOT NULL ,
    FOREIGN KEY (tipo) REFERENCES Tipologia(tipo)
);
```

```
CREATE TABLE Composizione_Treno (
    id_treno VARCHAR(5) NOT NULL ,
    id_vagone VARCHAR(5) NOT NULL ,
    posizione INT(1) NOT NULL ,
    PRIMARY KEY (id_treno, id_vagone) ,
    FOREIGN KEY (id_treno) REFERENCES Treno(id_treno) ,
    FOREIGN KEY (id_vagone) REFERENCES Vagone(id_vagone)
);
```

```
CREATE TABLE Corsa (
    id_corsa VARCHAR(5) PRIMARY KEY NOT NULL ,
    id_treno VARCHAR(5) NOT NULL ,
    id_stazione_p VARCHAR(5) NOT NULL ,
    id_stazione_a VARCHAR(5) NOT NULL ,
    ora_a TIME NOT NULL ,
    ora_p TIME NOT NULL ,
    FOREIGN KEY (id_treno) REFERENCES Treno(id_treno) ON DELETE CASCADE,
    FOREIGN KEY (id_stazione_a) REFERENCES Stazione(id_stazione),
    FOREIGN KEY (id_stazione_p) REFERENCES Stazione(id_stazione)
);
```

```
CREATE TABLE Biglietto (
    id_biglietto VARCHAR(5) PRIMARY KEY NOT NULL ,
    costo NUMERIC(4,2) NOT NULL
);
```

```
CREATE TABLE Associazione_Biglietto_Corsa (
    id_biglietto VARCHAR(5) NOT NULL ,
    id_corsa VARCHAR(5) NOT NULL ,
    PRIMARY KEY (id_biglietto, id_corsa) ,
    FOREIGN KEY (id_biglietto) REFERENCES Biglietto(id_biglietto) ,
    FOREIGN KEY (id_corsa) REFERENCES Corsa(id_corsa)
```

```
);
```

```
CREATE TABLE Cliente (  
    id_cliente VARCHAR(5) PRIMARY KEY NOT NULL ,  
    nome VARCHAR(30) NOT NULL ,  
    cognome VARCHAR(30) NOT NULL ,  
    email VARCHAR(30)  
);
```

```
CREATE TABLE Acquisto_Biglietto (  
    id_biglietto VARCHAR(5) NOT NULL ,  
    id_cliente VARCHAR(5) NOT NULL ,  
    data_acquisto DATE NOT NULL,  
    data_viaggio DATE NOT NULL,  
    PRIMARY KEY (id_biglietto, id_cliente) ,  
    FOREIGN KEY (id_biglietto) REFERENCES Biglietto(id_biglietto) ,  
    FOREIGN KEY (id_cliente) REFERENCES Cliente(id_cliente)  
);
```

Operazioni SQL

1. Stored Procedure per avere il numero totale di biglietti venduti mensilmente raggruppati per giorni :

```
CREATE PROCEDURE vendita_biglietti_mensile (IN mese INT(2))  
BEGIN  
    SELECT data_acquisto AS giorno,  
           COUNT(*) AS n°Biglietti  
    FROM acquisto_biglietto  
    WHERE MONTH(data_acquisto) = mese  
    GROUP BY data_acquisto  
    ORDER BY data_acquisto;  
END;  
  
SET @mese = 5;  
CALL vendita_biglietti_mensile(@mese);
```

2. Stored Procedure per avere una lista delle stazioni più trafficate nell'arco dell'anno:

```
CREATE PROCEDURE lista_stazioni_trafficate ()
BEGIN
    SELECT nome_stazione,
           COUNT(*) as traffico_persone
    FROM acquisto_biglietto
    INNER JOIN biglietto USING (id_biglietto)
    INNER JOIN associazione_biglietto_corsa USING (id_biglietto)
    INNER JOIN corsa USING (id_corsa)
    INNER JOIN stazione s on corsa.id_stazione_a = s.id_stazione
    GROUP BY id_stazione_a
    ORDER BY traffico_persone DESC;
END;

CALL lista_stazioni_trafficate();
```

3. Stored Procedure per avere una tabella con tutte le fermate e il rispettivo orario della fermata che un determinato treno effettuerà :

```
CREATE PROCEDURE fermate_treno (IN denom VARCHAR(30))
BEGIN
    SELECT CONCAT(nome_stazione, ' (', SUBSTRING(ora_a, 1, 5), ')') as
    Stazione_Arrivo_Ora
    FROM corsa c INNER JOIN stazione s on c.id_stazione_a = s.id_stazione
    WHERE id_treno = (SELECT id_treno FROM treno WHERE denominazione = denom)
    ORDER BY ora_a;
END;

SET @treno = 'Regionale 3562';
CALL fermate_treno(@treno);
```

4. Stored Procedure per avere i treni che sono in partenza da una stazione da un certo orario e che hanno come destinazione una determinata stazione :

```
CREATE PROCEDURE trova_treno (IN id_staz_a VARCHAR(5), IN id_staz_p VARCHAR(5), IN
tempo TIME)
BEGIN
    SELECT denominazione AS treno,
```

```

        SUBSTRING(c.ora_p, 1, 5) AS ora_partenza,
        SUBSTRING(c.ora_a, 1, 5) AS ora_arrivo
FROM corsa c
INNER JOIN treno t on c.id_treno = t.id_treno
WHERE id_staz_a = id_stazione_a AND id_stazione_p = id_staz_p AND ora_p > tempo
ORDER BY c.ora_p;
END;

```

```

SET @id_stazione_a = 'llxa2';
SET @id_stazione_p = '9o55i';
SET @tempo = '15:00';
CALL trova_treno(@id_stazione_a, @id_stazione_p, @tempo);

```

Osservazione : nonostante vengano inseriti gli id delle stazioni di arrivo e partenza, all'utente nell'applicazione web/sito verrà chiesto di inserire/scegliere le stazioni di arrivo e partenza.

5. Stored Procedure che per ogni treno mi da il numero di posti totali nel treno :

```

CREATE PROCEDURE num_posti_treni()
BEGIN
    SELECT denominazione,
           t.id_treno,
           SUM(num_posti) AS posti_Totali
    FROM treno t
    INNER JOIN composizione_treno ct on t.id_treno = ct.id_treno
    INNER JOIN vagone v on ct.id_vagone = v.id_vagone
    INNER JOIN tipologia tp on v.tipo = tp.tipo
    GROUP BY t.id_treno
    ORDER BY posti_Totali;
END;

CALL num_posti_treni();

```


6. Stored Procedure che mi da il riepilogo totale del biglietto con informazioni relative al cliente, al costo del biglietto e alle corse che il cliente deve prendere :

```
DELIMITER $$

CREATE PROCEDURE stampa_riepilogo (IN codice_b VARCHAR(5), IN codice_c VARCHAR(5),
OUT riepilogo VARCHAR(5000))

BEGIN

    DECLARE nominativo, denom_treno, ora_par, ora_arr, st1, st2 VARCHAR(30);
    DECLARE data_v DATE;
    DECLARE costo_biglietto DECIMAL(4,2);

    DECLARE finished INTEGER DEFAULT 0;
    DECLARE corse_cursor CURSOR FOR
        (SELECT denominazione,
            SUBSTRING(c.ora_p,1,5),
            s1.nome_stazione,
            SUBSTRING(c.ora_a,1,5),
            s2.nome_stazione
        FROM corsa c
        INNER JOIN associazione_biglietto_corsa USING (id_corsa)
        INNER JOIN treno USING (id_treno)
        INNER JOIN stazione s1 on c.id_stazione_p = s1.id_stazione
        INNER JOIN stazione s2 on c.id_stazione_a = s2.id_stazione
        WHERE id_biglietto = codice_b
        ORDER BY c.ora_a);

    DECLARE CONTINUE HANDLER FOR NOT FOUND SET finished = 1;

    SELECT CONCAT(nome,' ',cognome) INTO nominativo
    FROM cliente
    WHERE id_cliente = codice_c;

    SELECT costo INTO costo_biglietto
    FROM biglietto
    WHERE id_biglietto = codice_b;

    SELECT data_viaggio INTO data_v
    FROM acquisto_biglietto
```

```

WHERE id_cliente = codice_c AND id_biglietto = codice_b;

SET riepilogo = CONCAT(nominativo, '. Costo biglietto : ', costo_biglietto, ' euro.
Data Viaggio : ', data_v, ' ');

OPEN corse_cursor;

WHILE (finished = 0) DO
    FETCH corse_cursor INTO denom_treno, ora_par, st1, ora_arr, st2;
    IF finished = 0 THEN
        SET riepilogo = CONCAT(riepilogo, ' | ', denom_treno, ', ', st1, ' ->
        ', ora_par, ', ', st2, ' -> ', ora_arr);
    END IF;
END WHILE;

CLOSE corse_cursor;

END $$

DELIMITER ;

SET @codice_c = '15nci';
SET @codice_b = '1n7b3';
SET @stampa = '';
CALL stampa_riepilogo(@codice_b, @codice_c, @stampa);

```

7. Trigger per verificare che durante l'inserimento di una nuova corsa l'orario di arrivo non sia inferiore rispetto all'orario di partenza e che l'orario di arrivo non sia dopo mezzanotte :

```

DELIMITER $$

CREATE TRIGGER controllo_corse
BEFORE INSERT ON corsa FOR EACH ROW
BEGIN
    IF (NEW.ora_a < NEW.ora_p || NEW.ora_a > '24:00') THEN
        SIGNAL SQLSTATE '45001' SET MESSAGE_TEXT = 'Orario di arrivo o partenza
        invalidi';
    END IF;
END $$

DELIMITER ;

```